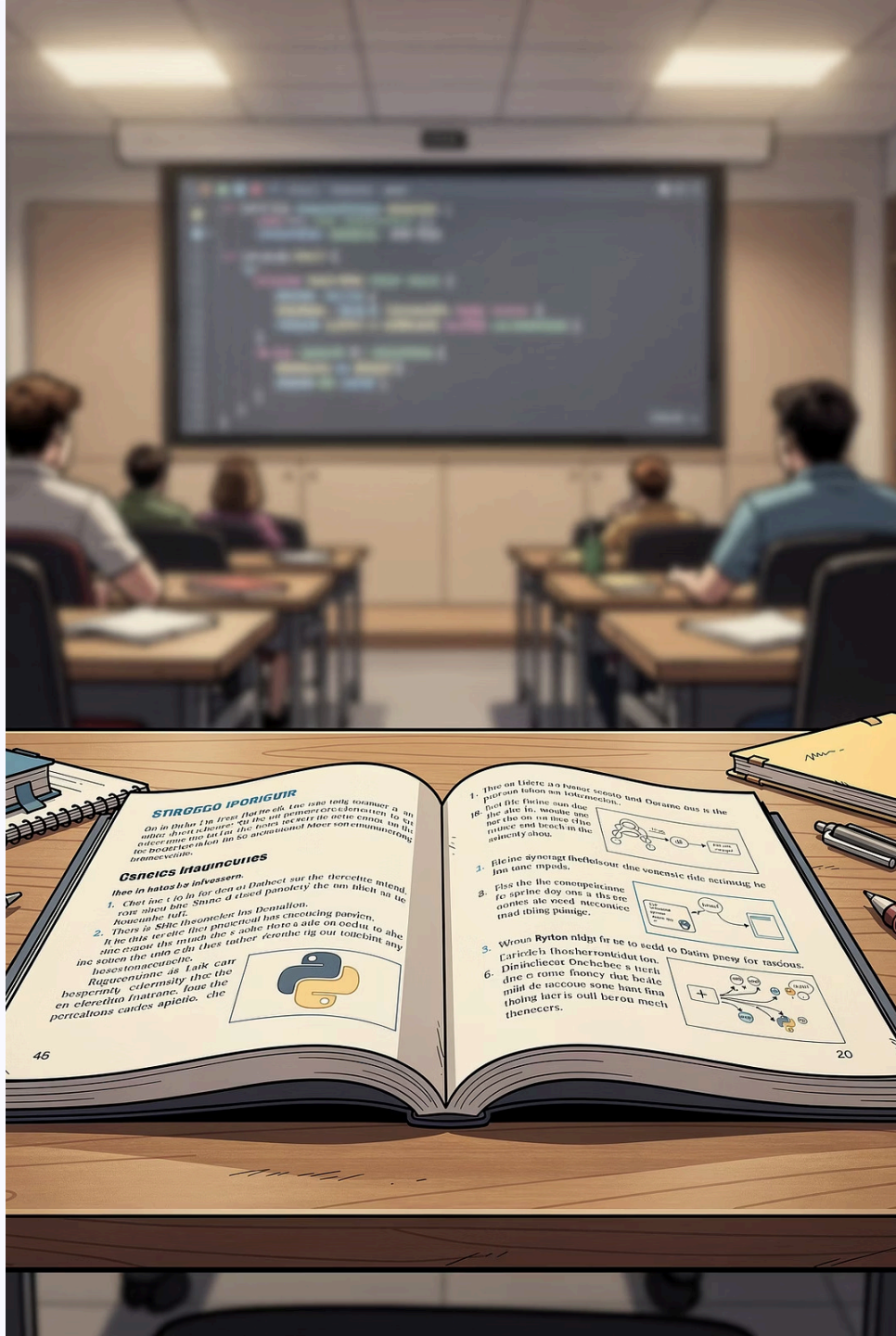
Chapter 3: Using Standard Libraries

Course 040223101 · Computer Programming for Mathematics 1
Department of Mathematics, Faculty of Applied Science, King Mongkut's University of Technology North Bangkok

Instructor: Assoc. Prof. Dr. Anuchit Jitpattanakul · Semester 1, Academic Year 2026



Learning Objectives

Upon completing this chapter, students will be able to:

01

Define Libraries & Modules

Explain the meaning and benefits of a **library** and a **module** in Python.

02

Import Modules

Use the `import` statement in its various forms to bring modules into a program.

03

Use the math Module

Apply functions and constants from the `math` module to solve mathematical problems.

04

Use the random Module

Generate random values with the `random` module for simulations and testing.

05

Choose the Right Library

Select and apply the appropriate library for a given problem context.

This chapter aligns with Course Learning Outcome **CLO 2**.

Chapter 3 Overview: Standard Libraries

One of Python's greatest strengths is its vast collection of **libraries** – pre-built collections of functions that you can call without writing them from scratch. This is especially valuable for mathematical work such as computing square roots, trigonometric functions, or generating random numbers.

This chapter introduces how to import and use the most important standard libraries, focusing on `math` and `random`.

3.1 What Are Libraries and Modules?

Module

A **module** is a Python file that groups related functions and constants together. For example, `math.py` contains mathematical tools.

Library / Package

When multiple modules are bundled together, the collection is called a **library** or **package**. Python ships with a **standard library** that is ready to use immediately after installation.

Why Use Libraries?

Libraries prevent you from rewriting code that others have already written and tested. This **saves time** and **reduces errors** – a core principle of good software engineering.

3.2 Importing Modules with `import`

Before using any module, you must import it. Python provides several import styles, each suited to different situations.

Style 1

`import module`

Import the whole module; prefix every call with the module name.

Style 2

`from module import name`

Import only specific items; call them directly without a prefix.

Style 3

`import module as alias`

Import the whole module under a short alias, e.g., `import numpy as np`.

3.2.1 Importing the Whole Module

The most straightforward approach: import the entire module and prefix every function call with the module name (e.g., `math.sqrt()`). This makes code easy to read because it is always clear where a function comes from.

File: 03import1.py

```
import math

print(math.sqrt(16))
print(math.pi)
```

Output

```
4.0
3.141592653589793
```

`math.sqrt(16)` returns the square root of 16 as a float. `math.pi` is the built-in constant π.

3.2.2 Importing Specific Items

Use `from ... import ...` to bring only the functions or constants you need into the current namespace. You can then call them **without** the module name prefix, making code more concise.

File: 03import2.py

```
from math import sqrt, pi

print(sqrt(25))
print(pi)
```

Output

```
5.0
3.141592653589793
```

Only `sqrt` and `pi` are imported. Other `math` functions remain unavailable unless explicitly imported.

3.2.3 Using an Alias with `as`

The `as` keyword lets you assign a shorter alias to a module. This is especially popular with libraries that have long names, such as `numpy` (aliased as `np`) or `matplotlib.pyplot` (aliased as `plt`).

File: 03import3.py

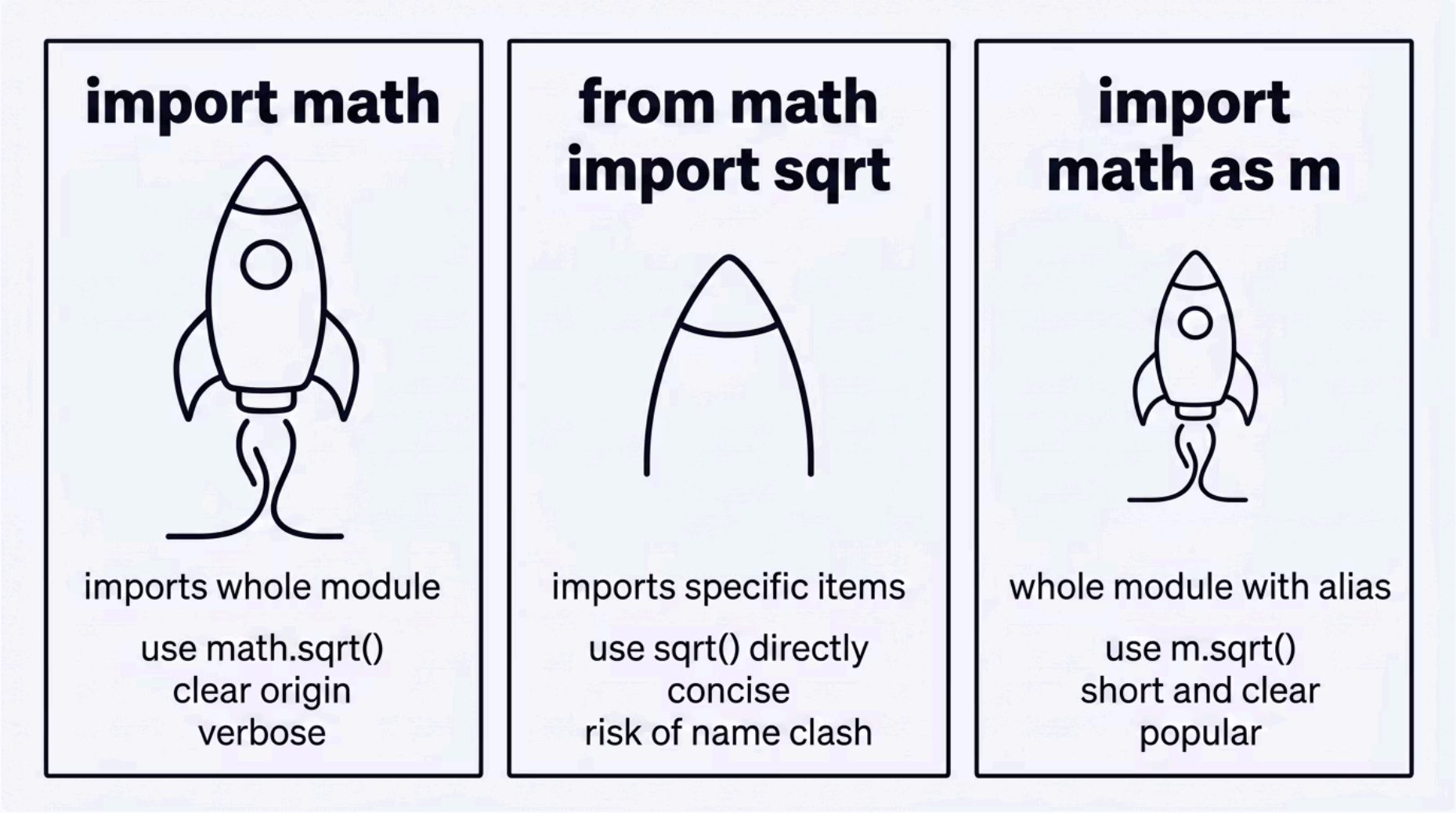
```
import math as m

print(m.factorial(5))
```

Output

```
120
```

Here `math` is aliased to `m`. Every call uses `m.` instead of `math.`, reducing typing while keeping the code readable.

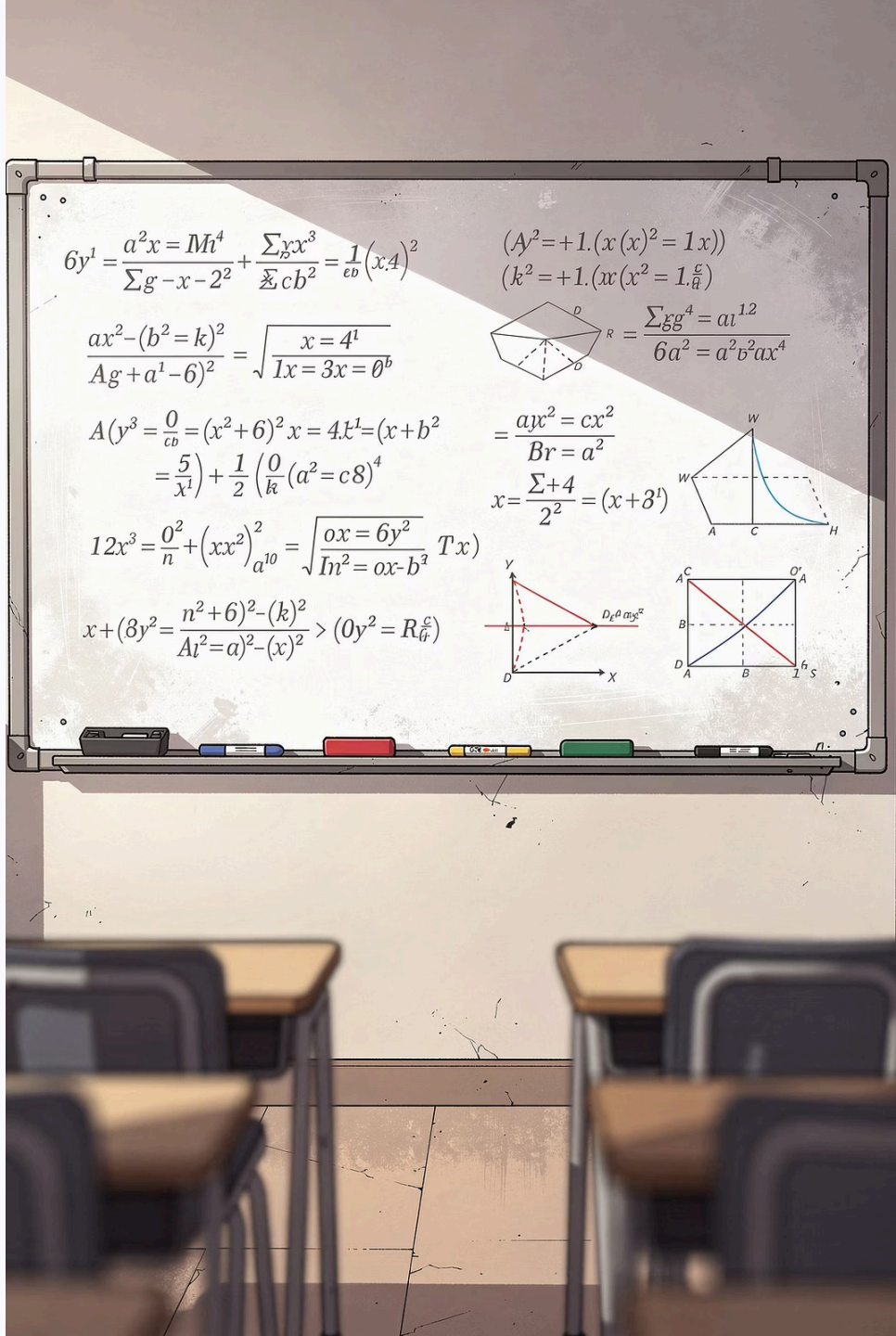
Comparing Import Styles



All three styles are valid. Choose based on readability and the risk of name conflicts in your program.

3.3 The `math` Module

The `math` module is part of Python's standard library and provides a comprehensive set of mathematical functions and constants. It is the go-to tool for numerical computations in mathematics courses.



math Module – Functions & Constants Reference

Function / Constant	Meaning	Example	Result
`math.sqrt(x)`	Square root of x	`math.sqrt(81)`	9.0
`math.pow(x, y)`	x raised to the power y	`math.pow(2, 5)`	32.0
`math.factorial(n)`	Factorial of n	`math.factorial(4)`	24
`math.floor(x)`	Round down to integer	`math.floor(4.7)`	4
`math.ceil(x)`	Round up to integer	`math.ceil(4.1)`	5
`math.gcd(a, b)`	Greatest common divisor of a and b	`math.gcd(12, 18)`	6
`math.log(x)`	Natural logarithm (base e)	`math.log(math.e)`	1.0
`math.sin(x)`	Sine (radians)	`math.sin(math.pi/2)`	1.0
`math.pi`	Constant π	`math.pi`	3.14159...
`math.e`	Constant e	`math.e`	2.71828...

Key math Functions at a Glance

sqrt & pow

`math.sqrt(x)` – square root, returns float.
`math.pow(x, y)` – x to the power y, also returns float.

Trigonometry

`math.sin(x)`, `math.cos(x)`, `math.tan(x)` – all accept angles in **radians**.

factorial

`math.factorial(n)` – computes n! for non-negative integers. E.g., `math.factorial(5)` = 120.

log & gcd

`math.log(x)` – natural log. `math.gcd(a, b)` – greatest common divisor of two integers.

Important: Trigonometric Functions Use Radians

⚠ Trigonometric functions in the `math` module accept angles in **radians**, not degrees. If your angle is in degrees, convert it first using `math.radians()`.

Correct Usage

```
import math
# Convert 30 degrees to radians
result = math.sin(math.radians(30))
print(result)  # 0.5
```

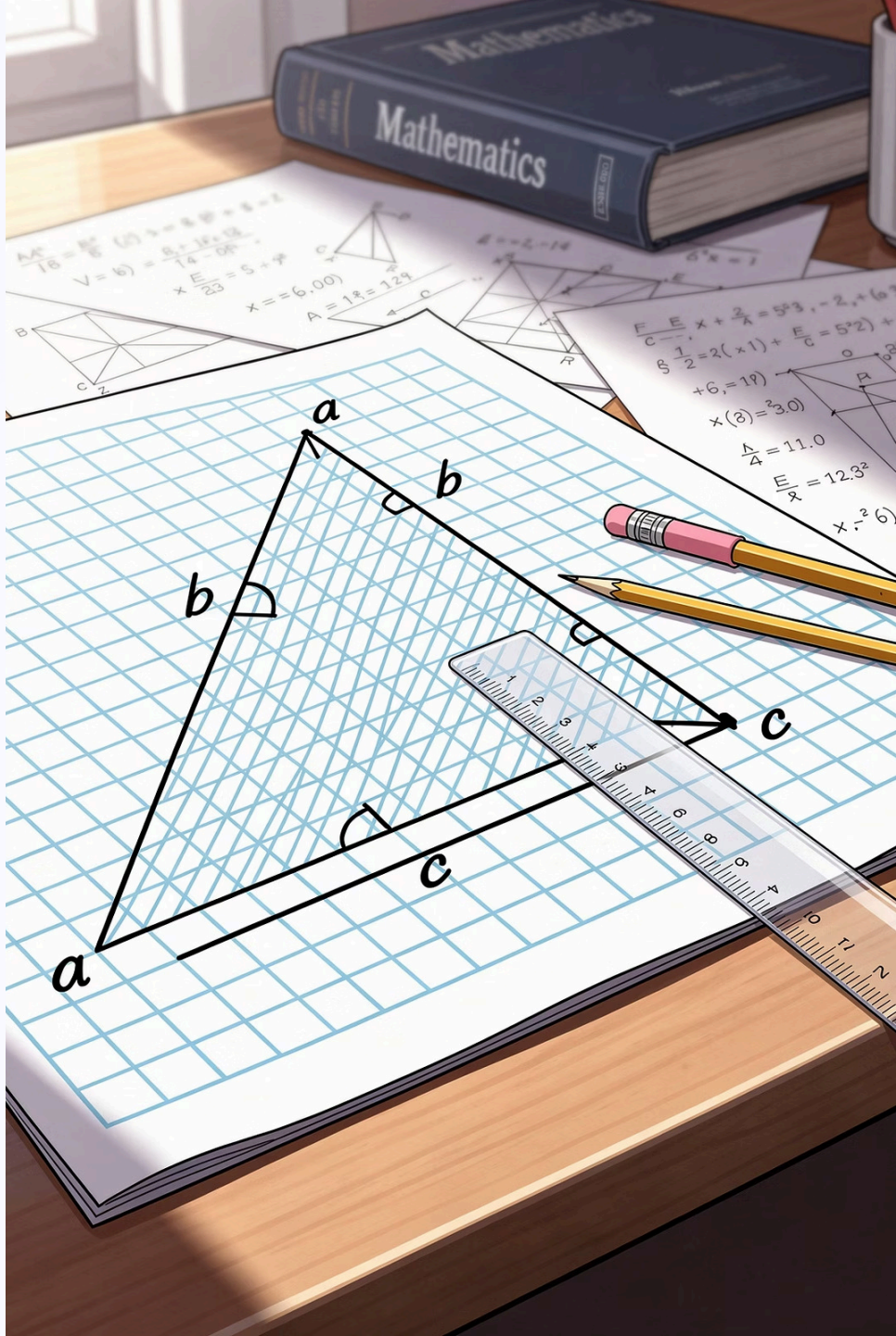
Why It Matters

Passing degrees directly to `math.sin()` will produce a wrong answer without any error message. For example, `math.sin(30)` gives approximately −0.988, not 0.5.

Always use `math.radians(angle_in_degrees)` as the argument when your input is in degrees.

Applied Example: Pythagorean Theorem

Using `math.sqrt()` to compute the hypotenuse of a right triangle given the two legs, applying the formula $c = \sqrt{a^2 + b^2}$.



File: 03pythagoras.py

Code

```
import math

a = 3
b = 4
c = math.sqrt(a ** 2 + b ** 2)
print(f"Hypotenuse = {c}")
```

Output

```
Hypotenuse = 5.0
```

Explanation

The program computes $c = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$.

The `**` operator raises a number to a power, and `math.sqrt()` returns the square root as a float. The classic 3-4-5 right triangle is confirmed.

This is a direct application of the Pythagorean theorem using a standard library function – no manual square root algorithm needed.

3.4 The `random` Module

The `random` module generates pseudo-random values. It is ideal for simulations, games, and creating test datasets.

Function	Meaning
`random.random()`	Random float in the range [0.0, 1.0)
`random.randint(a, b)`	Random integer in the range a to b (both endpoints included)
`random.uniform(a, b)`	Random float in the range a to b
`random.choice(seq)`	Randomly select one element from a sequence

Because values are random, the output will differ each time the program is run.

random Module – Code Example

File: 03random.py

```
import random

# Simulate rolling a die
print(random.randint(1, 6))

# Flip a coin
print(random.choice(["Heads", "Tails"]))
```

Sample Output

```
4
Tails
```

How It Works

`random.randint(1, 6)` returns a random integer between 1 and 6 inclusive – simulating a six-sided die roll.

`random.choice(["Heads", "Tails"])` randomly picks one element from the list – simulating a coin flip.

Because the values are pseudo-random, every run may produce different results. This is the expected and correct behavior.

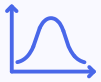
Use Cases for the random Module

Games & Simulations

Simulate dice rolls, card shuffles, or coin flips. Model probabilistic events in board games or casino simulations.

Test Data Generation

Create random datasets to test algorithms without needing real-world data. Useful for stress-testing programs.



Monte Carlo Methods

Use repeated random sampling to estimate mathematical quantities such as π or integrals – a powerful numerical technique.

3.5 Applied Example: Quadratic Equation Roots

Using the quadratic formula to find the roots of $ax^2 + bx + c = 0$ with the `math.sqrt()` function.

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

File: 03quadratic.py

Code

```
import math

a, b, c = 1, -5, 6

# Discriminant
d = b ** 2 - 4 * a * c

x1 = (-b + math.sqrt(d)) / (2 * a)
x2 = (-b - math.sqrt(d)) / (2 * a)

print(f"Roots: {x1} and {x2}")
```

Output

```
Roots: 3.0 and 2.0
```

Step-by-Step

For $x^2 - 5x + 6 = 0$ with a=1, b=−5, c=6:

1. Discriminant: $d = (-5)^2 - 4(1)(6) = 25 - 24 = 1$
2. Root 1: $x_1 = \frac{5+\sqrt{1}}{2} = 3.0$
3. Root 2: $x_2 = \frac{5-\sqrt{1}}{2} = 2.0$

The roots 3.0 and 2.0 are correct. You can verify: $(x-3)(x-2) = x^2-5x+6$ ✓

Discriminant and Root Cases

The discriminant $d = b^2 - 4ac$ determines the nature of the roots. The `math.sqrt()` function only works when $d \geq 0$.

d > 0

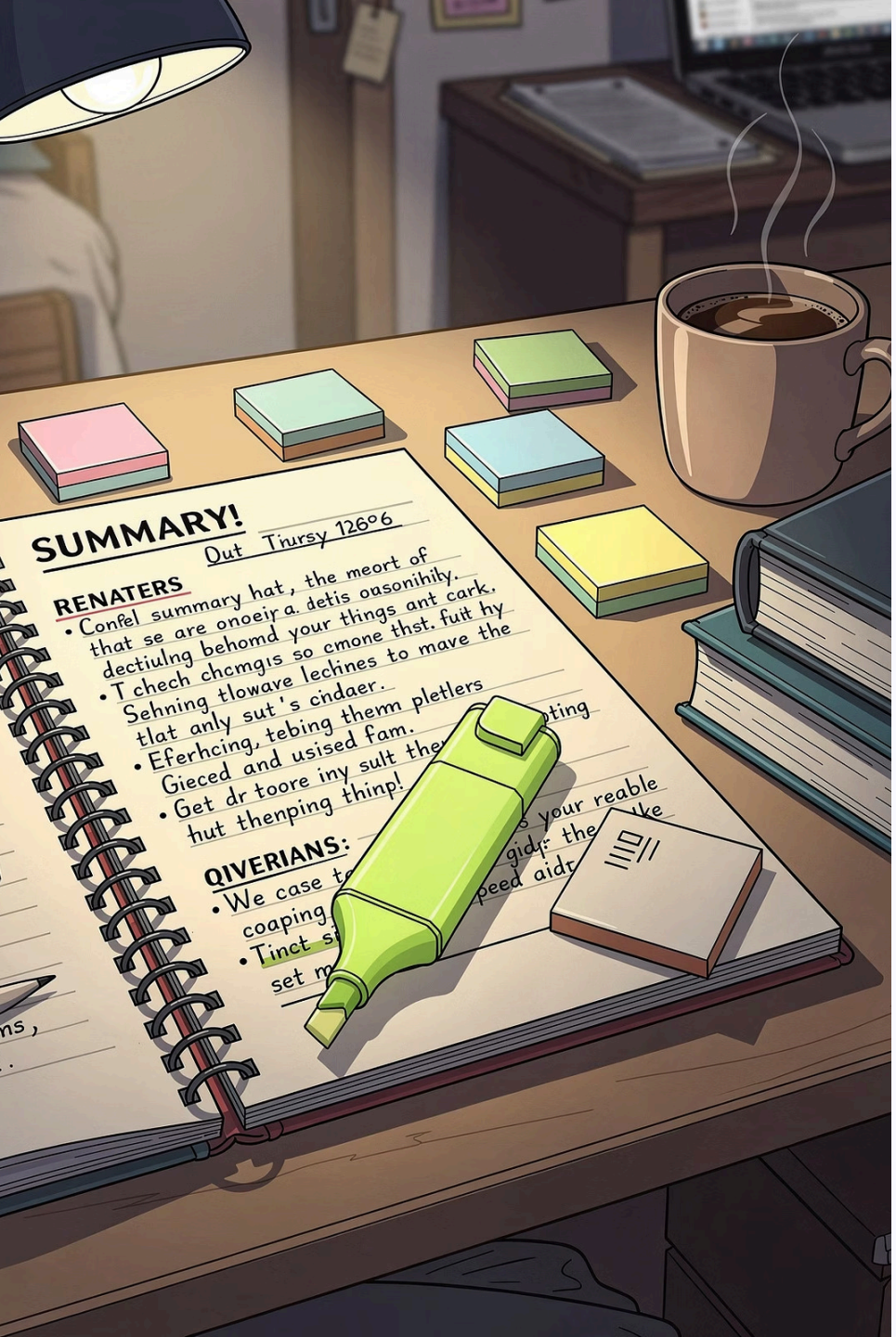
Two distinct real roots. `math.sqrt(d)` works normally.

d = 0

One repeated real root: $x = -b/(2a)$. Both x1 and x2 are equal.

d < 0

No real roots (complex roots). `math.sqrt(d)` raises a `ValueError`. A guard condition is needed.



3.6 Chapter Summary

Libraries Save Time

Libraries provide ready-made functions so you don't rewrite code that already exists. This reduces errors and speeds up development.

Three Import Styles

Use `import module`, `from module import name`, or `import module as alias` depending on readability and usage frequency.

math Module

Provides `sqrt`, `factorial`, `gcd`, trigonometric functions, and constants `pi` and `e`. Trig functions use radians.

random Module

Generates random integers, floats, and selections. Ideal for simulations, games, and test data generation.

Exercises – End of Chapter 3

Apply what you have learned by completing the following programming exercises.

1

Factorial Calculator

Write a program that reads a positive integer n from the user and displays the factorial of n using the `math` module.

2

Hypotenuse Finder

Write a program that reads the two legs of a right triangle and displays the length of the hypotenuse.

3

Two-Dice Simulator

Write a program that simulates rolling two dice and displays the value of each die and their sum.

4

GCD and LCM

Write a program that reads two integers and displays their GCD and LCM. Use the formula: LCM = a × b / GCD.

Exercise 1 – Solution: Factorial Calculator

Code

```
import math

n = int(input("Enter n: "))
print(f"{n}! = {math.factorial(n)}")
```

Sample Output

```
Enter n: 6
6! = 720
```

Explanation

`int(input(...))` reads a string from the user and converts it to an integer. `math.factorial(n)` computes n! directly.

For n = 6: $6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$

> The `math.factorial()` function only accepts non-negative integers. Passing a negative number or a float will raise a `ValueError`.

Exercise 3 – Solution: Two-Dice Simulator

Code

```
import random

d1 = random.randint(1, 6)
d2 = random.randint(1, 6)
print(f"Die 1 = {d1}, Die 2 = {d2}, Total = {d1 + d2}")
```

Sample Output

```
Die 1 = 5, Die 2 = 2, Total = 7
```

Explanation

`random.randint(1, 6)` is called twice – once for each die – returning a random integer from 1 to 6 inclusive each time.

The total is computed by simple addition. Because the values are random, the output will differ on every run.

> This is a classic Monte Carlo simulation in miniature. Extending this to thousands of rolls lets you estimate the probability distribution of sums.

Exercise 4 – Solution: GCD and LCM

Code

```
import math

a = int(input("a: "))
b = int(input("b: "))
g = math.gcd(a, b)
print(f"GCD = {g}, LCM = {a * b // g}")
```

Sample Output

```
a: 12
b: 18
GCD = 6, LCM = 36
```

Explanation

`math.gcd(12, 18)` returns 6, the largest integer that divides both 12 and 18 without remainder.

LCM is derived from the relationship: $\text{LCM}(a, b) = \frac{a \times b}{\gcd(a,b)}$

For a=12, b=18: $\text{LCM} = \frac{12 \times 18}{6} = 36$

The `//` operator performs integer (floor) division, ensuring the result is an integer.

Exercise 2 – Solution: Hypotenuse Finder

Code

```
import math

a = float(input("Leg a: "))
b = float(input("Leg b: "))
c = math.sqrt(a ** 2 + b ** 2)
print(f"Hypotenuse = {c}")
```

Sample Output

```
Leg a: 5
Leg b: 12
Hypotenuse = 13.0
```

Explanation

The program reads two leg lengths as floats, then applies the Pythagorean theorem: $c = \sqrt{a^2 + b^2}$.

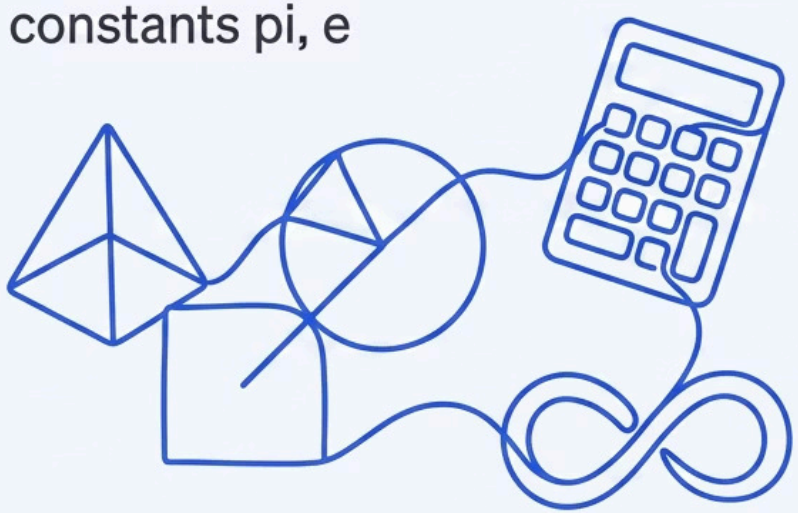
For a=5, b=12: $c = \sqrt{25 + 144} = \sqrt{169} = 13.0$

The 5-12-13 triple is another well-known Pythagorean triple, confirming the result is correct.

Comparing math vs. random Modules

MATH MODULE

COMPUTATION: sqrt, factorial, constants, constants pi, e



RANDOM MODULE

GENERATION: random(), random(), choice(), simulations

Python Standard Library – Broader Ecosystem

Beyond `math` and `random`, Python's standard library contains many other useful modules. Here are a few relevant to mathematics and computing:

statistics

Mean, median, mode, standard deviation, and variance for data sets.

fractions

Exact rational arithmetic using the `Fraction` class – no floating-point rounding errors.

decimal

High-precision decimal arithmetic, useful for financial and scientific calculations.

itertools

Combinatorial tools: permutations, combinations, Cartesian products, and more.

Third-Party Libraries for Mathematics

For advanced mathematical work, Python's ecosystem extends far beyond the standard library. These packages must be installed separately (e.g., via `pip install`).

NumPy

Fast numerical arrays, linear algebra, Fourier transforms, and matrix operations. The foundation of scientific Python.

Matplotlib

2D and 3D plotting library for creating graphs, charts, and mathematical visualizations.

SymPy

Symbolic mathematics: algebraic simplification, calculus, equation solving, and LaTeX output.

Best Practices When Using Libraries

Import at the Top

Place all `import` statements at the very top of your file, before any other code. This makes dependencies immediately visible to anyone reading the program.

Import Only What You Need

Avoid `from module import *` – it pollutes the namespace and makes it unclear where functions come from. Prefer explicit imports.

Use Meaningful Aliases

When aliasing, follow community conventions: `import numpy as np`, `import matplotlib.pyplot as plt`. This improves code readability across teams.

Check the Documentation

Python's official documentation at **docs.python.org** lists every function, its parameters, return type, and edge cases for all standard library modules.

Common Mistakes to Avoid

Forgetting to Import

Calling `sqrt(16)` without importing `math` raises a `NameError`. Always import before use.

Degrees vs. Radians

Passing degrees to `math.sin()` gives wrong results silently. Always convert with `math.radians()` first.

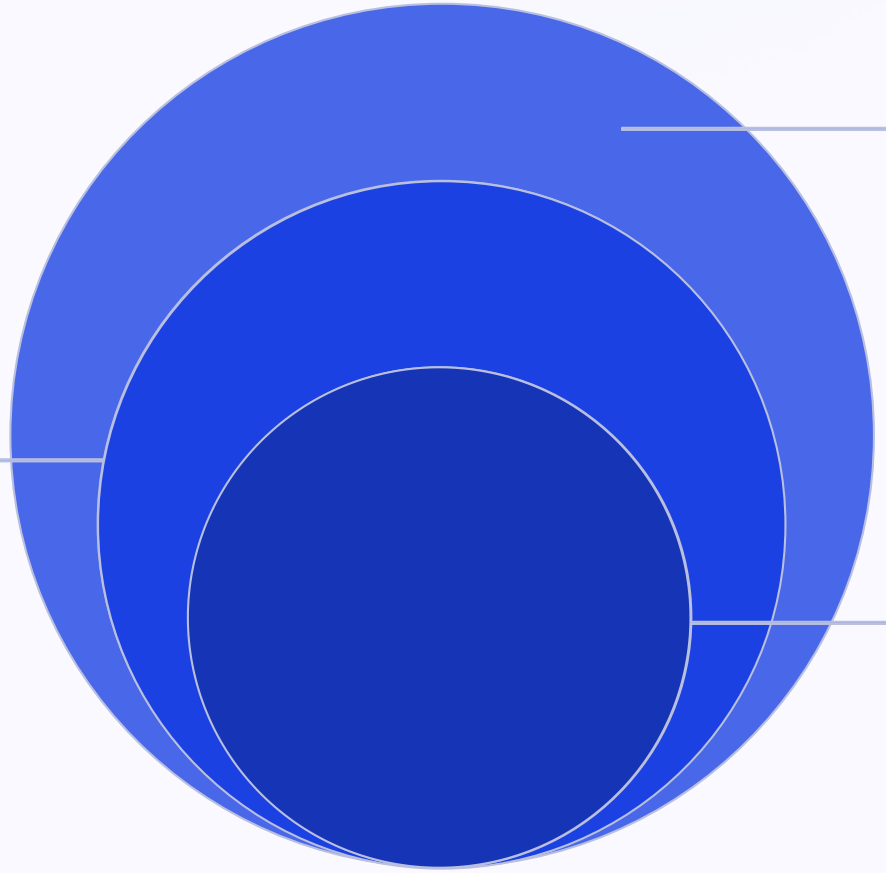
Negative Discriminant

Calling `math.sqrt(d)` when d < o raises a `ValueError`. Check the discriminant before computing roots.

Integer vs. Float

`math.sqrt()` always returns a float. `math.factorial()` returns an integer. Be aware of return types when combining results.

How Modules and Libraries Relate



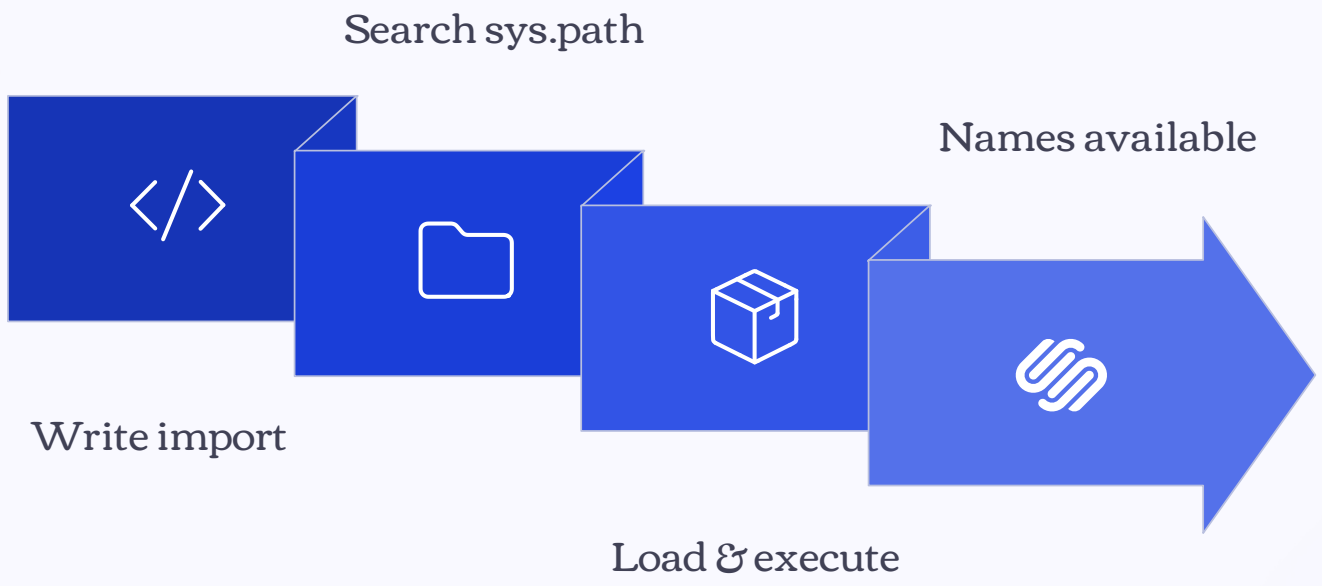
Standard Library
Built-in packages shipped with Python

Library / Package
Collection of modules organized as a package

Module
Single .py file with related functions

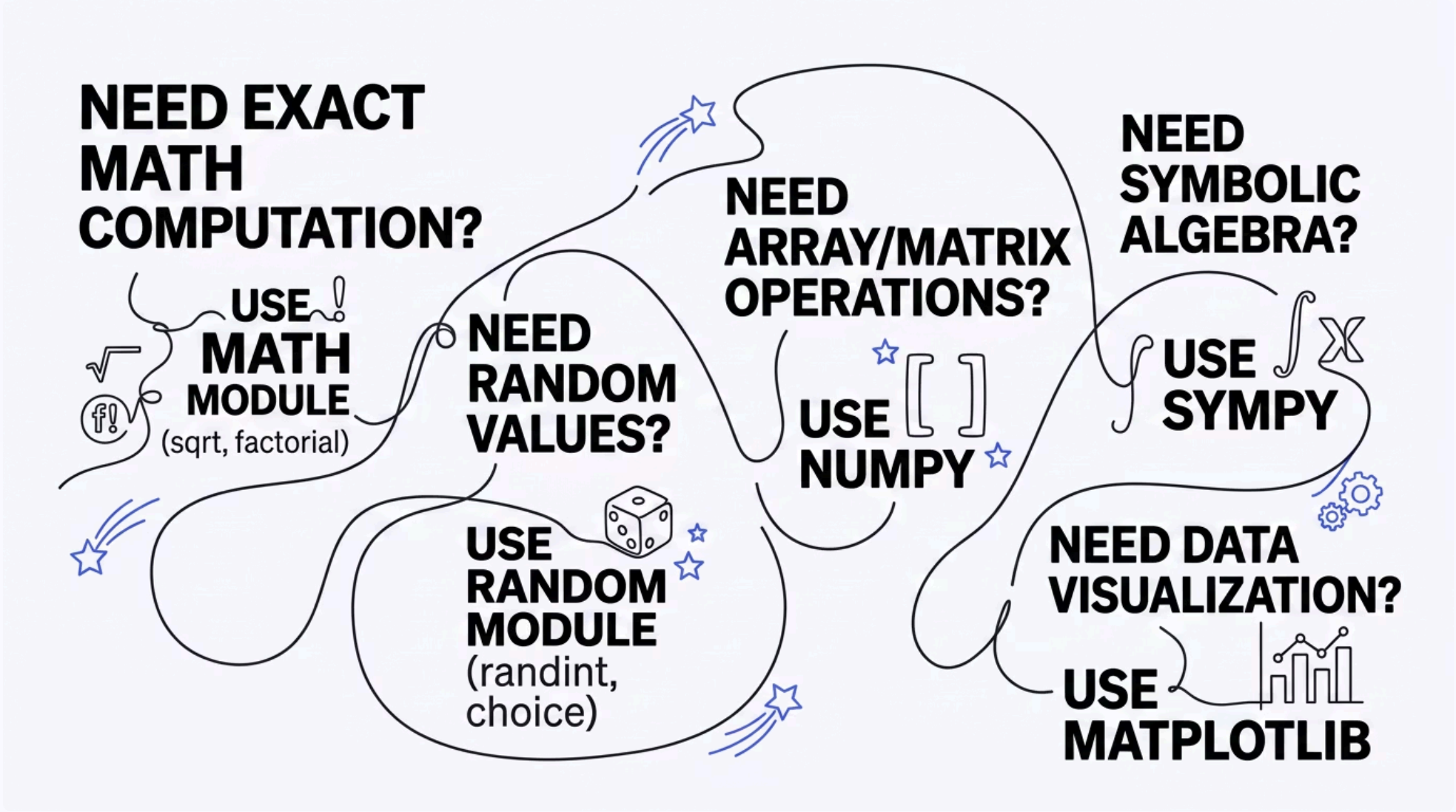
Understanding this hierarchy helps you navigate Python's documentation and know where to look when you need a specific function.

The Import Process – What Happens Behind the Scenes



When Python encounters an `import` statement, it searches a list of directories (`sys.path`) for the module file, loads it, and makes its contents available in your program's namespace.

Practical Workflow: Choosing the Right Tool



Quick Reference: math Module Constants

π

math.pi

3.141592653589793 – ratio of circumference to diameter of a circle.

e

math.e

2.718281828459045 – base of the natural logarithm.

τ

math.tau

6.283185307 – equal to 2π, the full angle in radians.

∞

math.inf

Positive infinity – useful for initializing minimum-search algorithms.

random Module – Deeper Look at randint vs. uniform

randint(a, b) – Integer Range

```
import random
# Returns 1, 2, 3, 4, 5, or 6
x = random.randint(1, 6)
print(x)
```

Both endpoints **a** and **b** are included. Returns an `int`.

uniform(a, b) – Float Range

```
import random
# Returns a float like 2.347...
x = random.uniform(1.0, 6.0)
print(x)
```

Returns a `float` anywhere in `[a, b]`. Useful for continuous simulations such as physical measurements or probability models.

Putting It All Together – A Mini Project Idea

Combine `math` and `random` to build a **Monte Carlo estimation of π**: generate random points in a unit square and check how many fall inside the unit circle. The ratio approximates π/4.

> This project uses `random.uniform()` for coordinates and `math.sqrt()` to compute distance – a perfect synthesis of both modules covered in this chapter.

Chapter 3 – Key Takeaways

Libraries = Reusable Code

Don't reinvent the wheel. Python's standard library provides tested, optimized functions for common tasks.

Three Import Styles

`import`, `from ... import`, and `import ... as` – each has its place depending on clarity and usage.

math for Computation

`sqrt`, `factorial`, `gcd`, trig functions, `pi`, and `e` – essential tools for mathematical programming.

random for Simulation

`randint`, `uniform`, `choice` – generate random values for games, simulations, and test data.

End of Chapter 3

Department of Mathematics · Faculty of Applied Science
King Mongkut's University of Technology North Bangkok

Course 040223101 · Computer Programming for Mathematics 1
Instructor: Assoc. Prof. Dr. Anuchit Jitpattanakul

You have completed Chapter 3: Using Standard Libraries. Proceed to the exercises to reinforce your understanding of the `math` and `random` modules.